

exploracao-beforeIT-pt2

September 7, 2026

1 Introdução

Este exercício tem dois propósitos: - Conhecer o funcionamento da simulação do Before.IT a partir das condições iniciais da parte 1. - Entender como as modificações que foram introduzidas para transferir dinheiro da população mais pobre para a mais rica afetam a simulação.

Para isso, vamos começar importando e iniciando o modelo.

```
import BeforeIT as Bit # importando o modelo
using DataFrames
using Plots
using StatsBase
using Random

Random.seed!(123)

model = Bit.Model(
    Bit.AUSTRIA2010Q1.parameters, # parametros das regras necessárias para
    ↪rodar o modelo
    Bit.AUSTRIA2010Q1.initial_conditions, # condições iniciais analisadas abaixo
);
```

1.1 Step de simulação e suas funções

Na implementação do Before.IT, função `Bit.step!` é responsável por rodar a simulação por um passo temporal, saindo do tempo `t` para o tempo `t+1`, sendo a unidade temporal um trimestre.

A função `Bit.step!` é composta por outras 48 funções que são chamadas sequencialmente. Iremos explorar estas funções uma por uma, explicando como elas alteram o estado do modelo.

1.1.1 Refinanciamento das empresas insolventes

A função `Bit.finance_insolvent_firms!(model)` identifica empresas que possuem simultaneamente depósitos (`D_i`) e patrimônio líquido negativos (`E_i`). Para cada empresa insolvente, o banco absorve a perda, recapitaliza a empresa, zera seus depósitos e ajusta sua dívida ao valor do capital usado como garantia.

Essa operação representa a resolução da insolvência antes do início das demais atividades do trimestre. Nas condições iniciais do modelo, nenhuma empresa está insolvente e, portanto, a função não provoca alterações.

```
firms = model.firms

ids_before = findall((firms.D_i .< 0) .& (firms.E_i .< 0))
bank_equity_before = model.bank.E_k

Bit.finance_insolvent_firms!(model)

ids_after = findall((firms.D_i .< 0) .& (firms.E_i .< 0))

DataFrame(
    moment = ["before", "after"],
    insolvent_firms = [length(ids_before), length(ids_after)],
    bank_equity = [bank_equity_before, model.bank.E_k],
)
```

	moment	insolvent_firms	bank_equity
	String	Int64	Float64
1	before	0	89460.0
2	after	0	89460.0

1.1.2 Formação das expectativas agregadas de crescimento e inflação

A função `Bit.set_growth_inflation_expectations!(model)` estima o PIB, o crescimento econômico e a inflação esperados para o próximo trimestre. Essa expectativa é compartilhada entre todos os agentes e será utilizada nos próximos passos.

Primeiro, define-se o logaritmo do PIB:

$$y_t = \log(Y_t)$$

O valor esperado é estimado por um processo autorregressivo AR(1):

$$y_t^e = \alpha_Y y_{t-1} + \beta_Y + \varepsilon_{Y,t}$$

Convertendo novamente para o nível do PIB:

$$Y_t^e = \exp(y_t^e)$$

A taxa de crescimento esperada é:

$$\gamma_t^e = \frac{Y_t^e}{Y_{t-1}} - 1$$

Para a inflação, o modelo também estima um processo AR(1):

$$q_t^e = \alpha_\pi q_{t-1} + \beta_\pi + \varepsilon_{\pi,t}$$

$$\pi_t^e = \exp(q_t^e) - 1$$

Os parâmetros α e β são estimados a partir dos dados históricos. Os termos ε representam choques aleatórios nas expectativas.

A função atualiza `model.agg.Y_e`, `model.agg.gamma_e` e `model.agg.pi_e`. Ela altera somente as expectativas; o PIB e a inflação efetivamente realizados ainda não são modificados.

```
expectations_before = (
    expected_gdp = model.agg.Y_e,
    expected_growth = model.agg.gamma_e,
    expected_inflation = model.agg.pi_e,
)

Bit.set_growth_inflation_expectations!(model)

DataFrame(
    moment = ["before", "after"],
    expected_gdp = [
        expectations_before.expected_gdp,
        model.agg.Y_e,
    ],
    expected_growth = [
        expectations_before.expected_growth,
        model.agg.gamma_e,
    ],
    expected_inflation = [
        expectations_before.expected_inflation,
        model.agg.pi_e,
    ],
)
```

	moment	expected_gdp	expected_growth	expected_inflation
	String	Float64	Float64	Float64
1	before	0.0	0.0	0.0
2	after	1.33838e5	-0.00592608	0.00223145

1.1.3 Como α e β são calculados?

Os parâmetros não são fixos. Eles são reestimados em cada trimestre usando todo o histórico disponível até aquele momento.

Para uma série $z_1, \dots, z_n$, o modelo estima a regressão autorregressiva:

$$z_t = \alpha z_{t-1} + \beta + u_t$$

A estimação usa os pares:

$$(z_1, z_2), (z_2, z_3), \dots, (z_{n-1}, z_n)$$

que podem ser escritos como:

$$\mathbf{y} = \begin{bmatrix} z_2 \\ z_3 \\ \vdots \\ z_n \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} z_1 & 1 \\ z_2 & 1 \\ \vdots & \vdots \\ z_{n-1} & 1 \end{bmatrix}$$

Os valores de α e β são escolhidos para minimizar a soma dos erros quadráticos:

$$(\hat{\alpha}, \hat{\beta}) = \underset{\alpha, \beta}{\operatorname{argmin}} \sum_{t=2}^n (z_t - \alpha z_{t-1} - \beta)^2$$

Em forma matricial:

$$\begin{bmatrix} \hat{\alpha} \\ \hat{\beta} \end{bmatrix} = \mathbf{X}^+ \mathbf{y}$$

onde $\mathbf{X}^+$ é a pseudoinversa de $\mathbf{X}$, calculada no modelo por decomposição em valores singulares, ou SVD.

A interpretação dos parâmetros é:

- $\hat{\alpha}$ mede a persistência da série;
- $\hat{\beta}$ é o intercepto ou deslocamento médio;
- u_t representa a parte que a regressão não conseguiu explicar.

Após a estimação, calcula-se a variância dos resíduos:

$$\hat{\sigma}_u^2 = \operatorname{Var}(\hat{u}_t)$$

e sorteia-se um novo choque:

$$\varepsilon_t \sim \mathcal{N}(0, \hat{\sigma}_u^2)$$

A previsão seguinte é então:

$$z_{n+1}^e = \hat{\alpha} z_n + \hat{\beta} + \varepsilon_t$$

Para o PIB, o modelo aplica esse procedimento sobre $z_t = \log(Y_t)$. O mesmo processo é aplicado à série histórica de inflação.

1.1.4 Sorteio dos choques externos

A próxima função, `Bit.set_epsilon!(model)`, sorteia três choques aleatórios:

- $\varepsilon_{Y,EA}$: choque sobre o PIB da área do euro;
- ε_E : choque sobre a demanda por exportações;
- ε_I : choque sobre a oferta de importações.

Durante a calibração, o modelo estima uma matriz de covariância $\mathbf{C}$ a partir dos resíduos históricos dessas três variáveis:

$$\mathbf{C} = \text{Cov}(\varepsilon_{Y,EA}, \varepsilon_E, \varepsilon_I)$$

Essa matriz preserva tanto a volatilidade individual de cada choque quanto suas correlações. Por exemplo, uma alteração no PIB da área do euro pode estar associada a uma alteração na demanda por exportações.

O modelo aplica a decomposição de Cholesky:

$$\mathbf{C} = \mathbf{L}\mathbf{L}^\top$$

e sorteia três números independentes de uma distribuição normal padrão:

$$\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Os choques correlacionados são obtidos por:

$$\varepsilon = \mathbf{L}\mathbf{z}$$

onde:

$$\varepsilon = \begin{bmatrix} \varepsilon_{Y,EA} \\ \varepsilon_E \\ \varepsilon_I \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \mathbf{C})$$

A função apenas sorteia e armazena os choques. Seus efeitos serão aplicados posteriormente ao PIB da área do euro, às exportações e às importações.

```
shocks_before = (  
    gdp_euro_area = model.agg.epsilon_Y_EA,  
    exports = model.agg.epsilon_E,  
    imports = model.agg.epsilon_I,  
)  
  
Bit.set_epsilon!(model)  
  
DataFrame(  
    shock = [  

```

```

    "PIB da área do euro",
    "Exportações",
    "Importações",
],
before = [
    shocks_before.gdp_euro_area,
    shocks_before.exports,
    shocks_before.imports,
],
after = [
    model.agg.epsilon_Y_EA,
    model.agg.epsilon_E,
    model.agg.epsilon_I,
],
)

```

	shock	before	after
	String	Float64	Float64
1	PIB da área do euro	0.0	-0.000343733
2	Exportações	0.0	0.0127531
3	Importações	0.0	0.00277058

1.1.5 Atualização da economia da área do euro

A função `Bit.set_growth_inflation_EA!(model)` atualiza o PIB, o crescimento e a inflação da área do euro.

O novo PIB é calculado por um processo AR(1):

$$Y_{EA,t} = \exp(\alpha_{Y,EA} \log(Y_{EA,t-1}) + \beta_{Y,EA} + \varepsilon_{Y,EA,t})$$

O crescimento realizado da área do euro é:

$$\gamma_{EA,t} = \frac{Y_{EA,t}}{Y_{EA,t-1}} - 1$$

Para a inflação, a função sorteia um choque independente:

$$\varepsilon_{\pi,EA,t} = \sigma_{\pi,EA} z_t, \quad z_t \sim \mathcal{N}(0, 1)$$

A nova inflação é calculada por:

$$\pi_{EA,t} = \exp(\alpha_{\pi,EA} \log(1 + \pi_{EA,t-1}) + \beta_{\pi,EA} + \varepsilon_{\pi,EA,t}) - 1$$

Os parâmetros α e β são estimados durante a calibração usando dados históricos trimestrais da área do euro. Diferentemente das expectativas domésticas, eles não são reestimados a cada período.

O choque $\varepsilon_{Y,EA,t}$ foi sorteado pela função anterior, enquanto o choque da inflação é sorteado nesta função.

```
euro_area_before = (
  gdp = model.rotw.Y_EA,
  growth = model.rotw.gamma_EA,
  inflation = model.rotw.pi_EA,
)

Bit.set_growth_inflation_EA!(model)

DataFrame(
  variable = ["GDP", "Growth", "Inflation"],
  before = [
    euro_area_before.gdp,
    euro_area_before.growth,
    euro_area_before.inflation,
  ],
  after = [
    model.rotw.Y_EA,
    model.rotw.gamma_EA,
    model.rotw.pi_EA,
  ],
)
```

	variable	before	after
	String	Float64	Float64
1	GDP	2.35485e6	2.35787e6
2	Growth	0.0	0.00128357
3	Inflation	0.00193832	-0.000740652

1.2 Atualização da taxa básica de juros

A função `Bit.set_central_bank_rate!(model)` determina a taxa básica de juros do novo trimestre por meio de uma regra de Taylor.

A taxa antes da restrição é:

$$\tilde{r}_t = \rho r_{t-1} + (1 - \rho) [r^* + \pi^* + \xi_\pi(\pi_{EA,t} - \pi^*) + \xi_\gamma \gamma_{EA,t}]$$

A taxa utilizada pelo modelo é limitada a valores não negativos:

$$r_t = \max(0, \tilde{r}_t)$$

Nessa equação:

- r_{t-1} é a taxa básica do trimestre anterior;
- ρ mede a persistência da taxa de juros;

- r^* é a taxa real de equilíbrio;
- π^* é a meta trimestral de inflação;
- $\pi_{EA,t}$ é a inflação atual da área do euro;
- $\gamma_{EA,t}$ é o crescimento atual da área do euro;
- ξ_π mede a reação do banco central à inflação;
- ξ_γ mede a reação ao crescimento econômico.

Quanto maior ρ , mais lentamente o banco central modifica a taxa. Quando a inflação fica acima da meta, o componente associado a ξ_π tende a elevar a taxa.

Durante a calibração, estima-se inicialmente:

$$r_t = \rho r_{t-1} + \gamma_1 \pi_{EA,t} + \gamma_2 \gamma_{EA,t} + u_t$$

Os pesos da regra de Taylor são obtidos por:

$$\xi_\pi = \frac{\gamma_1}{1 - \rho} \quad \text{e} \quad \xi_\gamma = \frac{\gamma_2}{1 - \rho}$$

A meta trimestral corresponde a uma inflação anual de 2%:

$$\pi^* = (1 + 0.02)^{1/4} - 1$$

A função atualiza apenas a taxa básica `model.cb.r_bar`. A taxa cobrada pelo banco comercial será atualizada posteriormente.

```
policy_rate_before = model.cb.r_bar

Bit.set_central_bank_rate!(model)

DataFrame(
  metric = [
    "Previous policy rate",
    "New policy rate",
    "Policy rate change",
  ],
  value = [
    policy_rate_before,
    model.cb.r_bar,
    model.cb.r_bar - policy_rate_before,
  ],
)
```

	metric	value
	String	Float64
1	Previous policy rate	0.00164593
2	New policy rate	0.00163333
3	Policy rate change	-1.25985e-5

1.2.1 Aplicação de um choque de cenário

Neste ponto, o modelo pode receber uma intervenção externa.

Representando o estado atual do modelo por $\mathcal{M}_t$ e o choque por S_t , temos:

$$\mathcal{M}_t^+ = S_t(\mathcal{M}_t)$$

Na simulação padrão, utiliza-se `NoShock`, definido por:

$$S_t(\mathcal{M}_t) = \mathcal{M}_t$$

Portanto, nenhuma variável é modificada.

Esse ponto também permite aplicar cenários alternativos, como uma taxa de juros fixa, uma mudança de produtividade ou uma alteração temporária da propensão ao consumo.

O choque é aplicado depois da regra de Taylor e antes da atualização da taxa do banco comercial. Assim, um choque de juros pode substituir a taxa definida anteriormente pelo Banco Central.

```
shock! = Bit.NoShock()

shock_result = shock!(model)

@assert shock_result === nothing
```

1.2.2 Atualização da taxa de juros do banco comercial

A função `Bit.set_bank_rate!(model)` calcula a taxa cobrada pelo banco comercial sobre empréstimos e posições financeiras negativas.

A taxa é definida por:

$$r_t = \bar{r}_t + \mu$$

onde:

- $\bar{r}_t$ é a taxa básica definida pelo Banco Central;
- μ é o prêmio de risco cobrado pelo banco comercial;
- r_t é a taxa bancária resultante.

O prêmio de risco é calculado durante a calibração como:

$$\mu = \frac{\text{juros pagos pelas empresas}}{\text{dívida total das empresas}} - \bar{r}$$

Assim, qualquer alteração da taxa básica é transmitida diretamente à taxa do banco, mantendo constante o prêmio de risco μ .

A função atualiza `model.bank.r`.

```

bank_rate_before = model.bank.r
policy_rate = model.cb.r_bar
risk_premium = model.prop.mu
expected_bank_rate = policy_rate + risk_premium

Bit.set_bank_rate!(model)

@assert isapprox(model.bank.r, expected_bank_rate)

DataFrame(
  metric = [
    "Previous bank rate",
    "Central bank policy rate",
    "Risk premium",
    "New bank rate",
  ],
  value = [
    bank_rate_before,
    policy_rate,
    risk_premium,
    model.bank.r,
  ],
)

```

	metric	value
	String	Float64
1	Previous bank rate	0.0283599
2	Central bank policy rate	0.00163333
3	Risk premium	0.026714
4	New bank rate	0.0283473

1.2.3 Expectativas e decisões das empresas

A função `Bit.set_firms_expectations_and_decisions!(model)` transforma as expectativas macroeconômicas em planos individuais para cada empresa.

Ela determina:

- a quantidade que a empresa pretende vender;
- o novo preço;
- o investimento desejado;
- a necessidade de insumos;
- o emprego desejado;
- o lucro esperado;
- o valor esperado do capital e da dívida;
- o novo crédito desejado.

Esses valores são planos. O crédito, as contratações e a produção efetiva acontecem nas etapas seguintes.

Quantidade planejada A empresa parte da demanda observada anteriormente e aplica o crescimento esperado:

$$Q_i^s = Q_i^d(1 + \gamma^e)$$

Inflação de custos O índice de custo dos insumos da empresa é:

$$P_i^M = \sum_s a_{s,G_i} \bar{P}_s$$

A pressão inflacionária causada pelos custos é:

$$\pi_i^c = (1 + \tau_{SIF}) \frac{\bar{w}_i}{\bar{\alpha}_i} \left(\frac{\bar{P}_{HH}}{P_i} - 1 \right) + \frac{1}{\beta_i} \left(\frac{P_i^M}{P_i} - 1 \right) + \frac{\delta_i}{\kappa_i} \left(\frac{\bar{P}_{CF}}{P_i} - 1 \right)$$

Os três componentes representam custos de trabalho, insumos intermediários e capital.

O novo preço combina a inflação de custos da empresa com a inflação agregada esperada:

$$P_i' = P_i(1 + \pi_i^c)(1 + \pi^e)$$

Necessidade de capital, insumos e trabalhadores A produção planejada é limitada pela capacidade do capital existente:

$$\tilde{Q}_i = \min(Q_i^s, K_i \kappa_i)$$

O investimento desejado é:

$$I_i^d = \frac{\delta_i}{\kappa_i} \tilde{Q}_i$$

A quantidade desejada de insumos intermediários é:

$$DM_i^d = \frac{\tilde{Q}_i}{\beta_i}$$

O emprego desejado é:

$$N_i^d = \max \left[1, \text{round} \left(\frac{\tilde{Q}_i}{\bar{\alpha}_i} \right) \right]$$

O limite inferior igual a um significa que cada empresa deseja manter pelo menos um trabalhador.

Lucro esperado O lucro anterior é ajustado pela inflação e pelo crescimento esperados:

$$\Pi_i^e = \Pi_i(1 + \pi^e)(1 + \gamma^e)$$

Varição esperada dos depósitos A variação esperada dos depósitos considera lucro, amortização da dívida, impostos e dividendos:

$$DD_i^e = \Pi_i^e - \theta L_i - \tau_{FIRM} \max(0, \Pi_i^e) - \theta_{DIV}(1 - \tau_{FIRM}) \max(0, \Pi_i^e)$$

Impostos e dividendos são pagos somente quando o lucro esperado é positivo.

Capital e dívida esperados O valor monetário esperado do capital é:

$$K_i^e = \bar{P}_{CF}(1 + \pi^e)K_i$$

A dívida esperada depois da amortização é:

$$L_i^e = (1 - \theta)L_i$$

Crédito desejado A empresa solicita crédito quando seus depósitos atuais, somados à variação esperada, não são suficientes:

$$DL_i^d = \max(0, -DD_i^e - D_i)$$

Portanto, se:

$$D_i + DD_i^e \geq 0,$$

a empresa não solicita um novo empréstimo.

Definição das variáveis

Símbolo	Código	Significado
i	índice	Empresa analisada
s	índice	Produto ou setor fornecedor
G_i	<code>G_i[i]</code>	Setor econômico da empresa
Q_i^d	<code>Q_d_i[i]</code>	Demanda anteriormente direcionada à empresa
Q_i^s	<code>Q_s_i[i]</code>	Quantidade planejada para venda
γ^e	<code>model.agg.gamma_e</code>	Crescimento agregado esperado
π^e	<code>model.agg.pi_e</code>	Inflação agregada esperada

Símbolo	Código	Significado
π_i^c	pi_c_i[i]	Inflação provocada pelos custos da empresa
P_i	P_i[i]	Preço anterior da empresa
P'_i	new_P_i[i]	Novo preço da empresa
$\bar{P}_{HH}$	P_bar_HH	Índice de preços do consumo das famílias
$\bar{P}_{CF}$	P_bar_CF	Índice de preços da formação de capital
$\bar{P}_s$	P_bar_g[s]	Índice de preços do produto do setor s
a_{s,G_i}	a_sg[s, G_i]	Participação do produto s nos insumos do setor da empresa
$\bar{w}_i$	w_bar_i[i]	Salário médio de referência
$\bar{\alpha}_i$	alpha_bar_i[i]	Produtividade normal do trabalho
β_i	beta_i[i]	Produtividade dos insumos intermediários
κ_i	kappa_i[i]	Produtividade do capital
δ_i	delta_i[i]	Taxa de depreciação do capital
K_i	K_i[i]	Estoque físico de capital
I_i^d	I_d_i[i]	Investimento desejado
DM_i^d	DM_d_i[i]	Insumos intermediários desejados
N_i^d	N_d_i[i]	Número desejado de trabalhadores
Π_i	Pi_i[i]	Lucro realizado anteriormente
Π_i^e	Pi_e_i[i]	Lucro esperado
D_i	D_i[i]	Depósitos ou liquidez da empresa
DD_i^e	variável intermediária	Variação esperada dos depósitos
L_i	L_i[i]	Dívida atual
L_i^e	L_e_i[i]	Dívida esperada depois da amortização
K_i^e	K_e_i[i]	Valor esperado do capital usado como garantia
DL_i^d	DL_d_i[i]	Novo empréstimo desejado
τ_{SIF}	tau_SIF	Contribuição social paga pelo empregador
τ_{FIRM}	tau_FIRM	Imposto sobre o lucro empresarial
θ	theta	Parcela trimestral de amortização da dívida
θ_{DIV}	theta_DIV	Parcela do lucro distribuída como dividendos

```

firms = model.firms

price_before = copy(firms.P_i)

Bit.set_firms_expectations_and_decisions!(model)

@assert all(firms.N_d_i .>= 1)
@assert all(firms.DL_d_i .>= 0)

firm_decisions = DataFrame(
    firm_id = firms.ID,
    sector = firms.G_i,
    target_quantity = firms.Q_s_i,
    price_before = price_before,
    new_price = firms.P_i,
    desired_investment = firms.I_d_i,
    desired_materials = firms.DM_d_i,
    desired_employment = firms.N_d_i,
    expected_profit = firms.Pi_e_i,
    desired_new_loans = firms.DL_d_i,
    expected_capital = firms.K_e_i,
    expected_loans = firms.L_e_i,
)

first(firm_decisions, 10)

```

	firm_id	sector	target_quantity	price_before	new_price	desired_investment	desired_materials
	Int64	Int64	Float64	Float64	Float64	Float64	Float64
1	1	1	32.2737	1.0	1.00223	8.37302	19.4049
2	2	1	10.7579	1.0	1.00223	2.79101	6.46829
3	3	1	10.7579	1.0	1.00223	2.79101	6.46829
4	4	1	10.7579	1.0	1.00223	2.79101	6.46829
5	5	1	10.7579	1.0	1.00223	2.79101	6.46829
6	6	1	10.7579	1.0	1.00223	2.79101	6.46829
7	7	1	10.7579	1.0	1.00223	2.79101	6.46829
8	8	1	21.5158	1.0	1.00223	5.58201	12.9366
9	9	1	10.7579	1.0	1.00223	2.79101	6.46829
10	10	1	10.7579	1.0	1.00223	2.79101	6.46829

1.2.4 Mercado de crédito

A função `Bit.search_and_matching_credit!(model)` determina quanto crédito cada empresa efetivamente recebe.

Primeiro, são selecionadas as empresas que desejam novos empréstimos:

$$\mathcal{J}_{FG} = \{i : \Delta L_i^d > 0\}$$

A ordem dessas empresas é embaralhada aleatoriamente. Em seguida, o banco avalia cada empresa de maneira sequencial.

O crédito concedido à empresa i é:

$$\Delta L_i = \max \left[0, \min \left(\Delta L_i^d, \zeta_{LTV} K_i^e - L_i^e, \frac{E_k}{\zeta} - \sum_j L_j^e - S_{\Delta L} \right) \right]$$

Essa expressão aplica três limites ao empréstimo:

1. Demanda da empresa

$$\Delta L_i \leq \Delta L_i^d$$

A empresa não recebe mais crédito do que solicitou.

2. Garantia disponível

$$L_i^e + \Delta L_i \leq \zeta_{LTV} K_i^e$$

A dívida total não pode ultrapassar uma fração do valor esperado do capital da empresa.

3. Capacidade do banco

$$\sum_j L_j^e + \sum_j \Delta L_j \leq \frac{E_k}{\zeta}$$

O total de empréstimos não pode ultrapassar a capacidade permitida pelo patrimônio do banco.

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa avaliada
$\mathcal{J}_{FG}$	I_FG	Conjunto de empresas que solicitaram crédito
ΔL_i^d	DL_d_i[i]	Novo empréstimo desejado pela empresa
ΔL_i	DL_i[i]	Novo empréstimo efetivamente concedido
K_i^e	K_e_i[i]	Valor esperado do capital usado como garantia
L_i^e	L_e_i[i]	Dívida esperada após a amortização programada
ζ_{LTV}	zeta_LTV	Limite da dívida em relação ao valor da garantia
E_k	E_k	Patrimônio líquido do banco

Símbolo	Código	Significado
ζ	zeta	Exigência de capital próprio do banco
E_k/ζ	—	Volume máximo de crédito sustentado pelo banco
$\sum_j L_j^e$	s_L_e	Total das dívidas esperadas antes dos novos empréstimos
$S_{\Delta L}$	s_DL	Crédito novo já concedido às empresas anteriores

Como as empresas são atendidas em ordem aleatória, pode ocorrer racionamento de crédito. Quando a capacidade do banco é insuficiente, as primeiras empresas avaliadas podem receber crédito enquanto as últimas recebem apenas uma parte ou nada.

A função armazena o crédito concedido em `model.firms.DL_i`. A dívida total das empresas ainda não é atualizada nesta etapa.

```
firms = model.firms

desired_loans = copy(firms.DL_d_i)
collateral_headroom = max(
    0.0,
    model.prop.zeta_LTV .* firms.K_e_i .- firms.L_e_i,
)

bank_credit_capacity = model.bank.E_k / model.prop.zeta
existing_expected_loans = sum(firms.L_e_i)
bank_credit_headroom = max(
    0.0,
    bank_credit_capacity - existing_expected_loans,
)

Bit.search_and_matching_credit!(model)

credit_allocation = DataFrame(
    firm_id = firms.ID,
    desired_loan = desired_loans,
    collateral_headroom = collateral_headroom,
    actual_loan = firms.DL_i,
)

DataFrame(
    metric = [
        "Bank credit capacity",
        "Existing expected loans",
        "Available credit before allocation",
        "Total desired loans",
    ]
)
```



```

        "Total granted loans",
    ],
    value = [
        bank_credit_capacity,
        existing_expected_loans,
        bank_credit_headroom,
        sum(desired_loans),
        sum(firms.DL_i),
    ],
)

```

	metric	value
	String	Float64
1	Bank credit capacity	2.982e6
2	Existing expected loans	225073.0
3	Available credit before allocation	2.75693e6
4	Total desired loans	4104.33
5	Total granted loans	4104.33

1.2.5 Mercado de trabalho

A função `Bit.search_and_matching_labour!(model)` ajusta o número de trabalhadores de cada empresa por meio de demissões e contratações.

Para cada empresa, calcula-se a diferença entre o emprego desejado e o emprego atual:

$$V_i = N_i^d - N_i$$

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa analisada
h	índice do trabalhador	Trabalhador analisado
N_i^d	<code>N_d[i]</code>	Número de trabalhadores desejado pela empresa
N_i	<code>N_i[i]</code>	Número atual de trabalhadores da empresa
V_i	<code>V_i[i]</code>	Número de vagas, quando positivo, ou excesso de trabalhadores, quando negativo
O_h	<code>O_h[h]</code>	Situação ocupacional do trabalhador
$\mathcal{H}_E$	<code>H_E</code>	Conjunto de trabalhadores empregados
$\mathcal{H}_U$	<code>H_U</code>	Conjunto de trabalhadores desempregados

Símbolo	Código	Significado
$\mathcal{J}_V$	I_V	Conjunto de empresas com vagas abertas

A variável de ocupação é interpretada como:

$$O_h = \begin{cases} 0, & \text{se o trabalhador está desempregado,} \\ i, & \text{se o trabalhador está empregado na empresa } i. \end{cases}$$

Demissões Quando:

$$V_i < 0,$$

a empresa possui mais trabalhadores do que deseja. Os trabalhadores empregados são visitados em ordem aleatória e demitidos até que o excesso seja eliminado.

Para cada demissão:

$$O_h \leftarrow 0$$

$$N_i \leftarrow N_i - 1$$

$$V_i \leftarrow V_i + 1$$

Contratações Depois das demissões, o modelo identifica os trabalhadores desempregados:

$$\mathcal{H}_U = \{h : O_h = 0\}$$

e as empresas que ainda possuem vagas:

$$\mathcal{J}_V = \{i : V_i > 0\}$$

Para cada contratação:

$$O_h \leftarrow i$$

$$N_i \leftarrow N_i + 1$$

$$V_i \leftarrow V_i - 1$$

O processo termina quando não existem mais trabalhadores desempregados ou empresas com vagas:

$$\mathcal{H}_U = \emptyset \quad \text{ou} \quad \mathcal{J}_V = \emptyset$$

O pareamento é aleatório: o modelo não considera qualificação, salário, experiência ou localização. Ao final, são atualizados o emprego das empresas, N_i , e a ocupação dos trabalhadores, O_h .

O vetor V_i é temporário e não é armazenado no estado do modelo.

```
firms = model.firms

employment_before = copy(firms.N_i)
occupation_before = copy(model.w_act.O_h)
desired_employment = copy(firms.N_d_i)

Bit.search_and_matching_labour!(model)

@assert sum(firms.N_i) == count(>(0), model.w_act.O_h)

firm_employment = DataFrame(
    firm_id = firms.ID,
    desired_employment = desired_employment,
    employment_before = employment_before,
    employment_after = firms.N_i,
    remaining_gap = desired_employment .- firms.N_i,
)

DataFrame(
    metric = [
        "Total employment before",
        "Total employment after",
        "Unemployed workers before",
        "Unemployed workers after",
        "Remaining vacancies",
    ],
    value = [
        sum(employment_before),
        sum(firms.N_i),
        count(==(0), occupation_before),
        count(==(0), model.w_act.O_h),
        sum(max.(0, desired_employment .- firms.N_i)),
    ],
)
```

	metric	value
	String	Int64
1	Total employment before	3866
2	Total employment after	3864
3	Unemployed workers before	252
4	Unemployed workers after	254
5	Remaining vacancies	0

1.2.6 Definição dos salários pelas empresas

A função `Bit.set_firms_wages!(model)` determina o salário oferecido por cada empresa depois das contratações e demissões.

Primeiro, calcula-se a produção que a empresa deseja realizar e consegue sustentar com seu capital e seus insumos:

$$\tilde{Y}_i = \min(Q_i^s, K_i \kappa_i, M_i \beta_i)$$

Em seguida, compara-se essa produção com a capacidade normal dos trabalhadores empregados:

$$m_i = \min\left(1.5, \frac{\tilde{Y}_i}{N_i \bar{\alpha}_i}\right)$$

O novo salário da empresa é:

$$w_i = \bar{w}_i m_i$$

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa analisada
w_i	<code>w_i[i]</code>	Novo salário definido pela empresa
$\bar{w}_i$	<code>w_bar_i[i]</code>	Salário médio de referência da empresa
m_i	<code>wage_multiplier[i]</code>	Fator de ajuste aplicado ao salário de referência
$\tilde{Y}_i$	<code>feasible_output[i]</code>	Produção desejada que capital e insumos conseguem sustentar
Q_i^s	<code>Q_s_i[i]</code>	Quantidade que a empresa planeja produzir ou vender
K_i	<code>K_i[i]</code>	Estoque de capital da empresa
κ_i	<code>kappa_i[i]</code>	Produtividade do capital
$K_i \kappa_i$	<code>K_i[i] * kappa_i[i]</code>	Capacidade produtiva proporcionada pelo capital

Símbolo	Código	Significado
M_i	<code>M_i[i]</code>	Estoque de insumos intermediários
β_i	<code>beta_i[i]</code>	Produtividade dos insumos intermediários
$M_i\beta_i$	<code>M_i[i] * beta_i[i]</code>	Capacidade produtiva proporcionada pelos insumos
N_i	<code>N_i[i]</code>	Número atual de trabalhadores da empresa
$\bar{\alpha}_i$	<code>alpha_bar_i[i]</code>	Produtividade normal de cada trabalhador
$N_i\bar{\alpha}_i$	—	Capacidade produtiva normal do trabalho

Interpretação do fator de ajuste Quando:

$$m_i < 1,$$

o salário fica abaixo do valor de referência porque a empresa possui capacidade de trabalho superior à produção planejada.

Quando:

$$m_i = 1,$$

o salário é igual ao valor de referência.

Quando:

$$1 < m_i \leq 1.5,$$

o salário fica acima do valor de referência. O limite de 1.5 impede que ele ultrapasse 150% do salário médio da empresa.

Essa função atualiza `model.firms.w_i`. Os salários individuais dos trabalhadores, `model.w_act.w_h`, serão atualizados em uma etapa posterior.

```
firms = model.firms

wage_before = copy(firms.w_i)

feasible_output = min.(
    firms.Q_s_i,
    firms.K_i .* firms.kappa_i,
    firms.M_i .* firms.beta_i,
)
```

```

labour_capacity = firms.N_i .* firms.alpha_bar_i

wage_multiplier = min.(
    1.5,
    feasible_output ./ labour_capacity,
)

expected_wage = firms.w_bar_i .* wage_multiplier

Bit.set_firms_wages!(model)

@assert all(isapprox.(firms.w_i, expected_wage))

firm_wages = DataFrame(
    firm_id = firms.ID,
    reference_wage = firms.w_bar_i,
    wage_before = wage_before,
    feasible_output = feasible_output,
    labour_capacity = labour_capacity,
    wage_multiplier = wage_multiplier,
    new_wage = firms.w_i,
)

first(firm_wages, 10)

```

	firm_id	reference_wage	wage_before	feasible_output	labour_capacity	wage_multiplier	
	Int64	Float64	Float64	Float64	Float64	Float64	
1	1	0.26971	0.0	32.2737	32.4661	0.994074	...
2	2	0.26971	0.0	10.7579	10.822	0.994074	...
3	3	0.26971	0.0	10.7579	10.822	0.994074	...
4	4	0.26971	0.0	10.7579	10.822	0.994074	...
5	5	0.26971	0.0	10.7579	10.822	0.994074	...
6	6	0.26971	0.0	10.7579	10.822	0.994074	...
7	7	0.26971	0.0	10.7579	10.822	0.994074	...
8	8	0.26971	0.0	21.5158	21.6441	0.994074	...
9	9	0.26971	0.0	10.7579	10.822	0.994074	...
10	10	0.26971	0.0	10.7579	10.822	0.994074	...

1.2.7 Produção das empresas

A função `Bit.set_firms_production!(model)` calcula quanto cada empresa efetivamente produz no trimestre.

Produtividade efetiva do trabalho Primeiro, calcula-se a produção desejada que pode ser sustentada pelo capital e pelos insumos:

$$\widetilde{Y}_i = \min(Q_i^s, K_i \kappa_i, M_i \beta_i)$$

A produtividade efetiva do trabalho é:

$$\alpha_i = \bar{\alpha}_i \min \left(1.5, \frac{\tilde{Y}_i}{N_i \bar{\alpha}_i} \right)$$

O limite de 1.5 permite que a produtividade efetiva alcance no máximo 150% da produtividade normal.

Função de produção de Leontief A produção realizada é determinada pelo recurso mais restritivo:

$$Y_i = \min (Q_i^s, N_i \alpha_i, K_i \kappa_i, M_i \beta_i)$$

Essa é uma função de produção de Leontief. Trabalho, capital e insumos são complementares: uma quantidade maior de apenas um recurso não aumenta a produção quando outro recurso constitui o gargalo.

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa analisada
Y_i	<code>Y_i[i]</code>	Produção efetivamente realizada
Q_i^s	<code>Q_s_i[i]</code>	Quantidade que a empresa planeja produzir ou vender
$\tilde{Y}_i$	<code>feasible_output[i]</code>	Produção desejada limitada por capital e insumos
N_i	<code>N_i[i]</code>	Número de trabalhadores empregados
$\bar{\alpha}_i$	<code>alpha_bar_i[i]</code>	Produtividade normal de cada trabalhador
α_i	<code>effective_productivity[i]</code>	Produtividade efetiva de cada trabalhador
$N_i \alpha_i$	<code>labour_output[i]</code>	Capacidade produtiva do trabalho
K_i	<code>K_i[i]</code>	Estoque de capital
κ_i	<code>kappa_i[i]</code>	Produtividade do capital
$K_i \kappa_i$	<code>capital_output[i]</code>	Capacidade produtiva do capital
M_i	<code>M_i[i]</code>	Estoque de insumos intermediários
β_i	<code>beta_i[i]</code>	Produtividade dos insumos
$M_i \beta_i$	<code>material_output[i]</code>	Capacidade produtiva dos insumos

A função atualiza `model.firms.Y_i`. Essa produção ainda não corresponde às vendas: a quantidade vendida será determinada posteriormente no mercado de bens.

```
firms = model.firms

@assert all(firms.N_i .> 0)

production_before = copy(firms.Y_i)

capital_output = firms.K_i .* firms.kappa_i
material_output = firms.M_i .* firms.beta_i

feasible_output = min.(
    firms.Q_s_i,
    capital_output,
    material_output,
)

productivity_multiplier = min.(
    1.5,
    feasible_output ./ (firms.N_i .* firms.alpha_bar_i),
)

effective_productivity =
    firms.alpha_bar_i .* productivity_multiplier

labour_output = firms.N_i .* effective_productivity

expected_production = min.(
    firms.Q_s_i,
    labour_output,
    capital_output,
    material_output,
)

Bit.set_firms_production!(model)

@assert all(isapprox.(firms.Y_i, expected_production))

firm_production = DataFrame(
    firm_id = firms.ID,
    target_quantity = firms.Q_s_i,
    production_before = production_before,
    labour_output = labour_output,
    capital_output = capital_output,
    material_output = material_output,
    actual_production = firms.Y_i,
```



```
)
first(firm_production, 10)
```

	firm_id	target_quantity	production_before	labour_output	capital_output	material_output	
	Int64	Float64	Float64	Float64	Float64	Float64	
1	1	32.2737	32.4661	32.2737	38.1954	38.1954	...
2	2	10.7579	10.822	10.7579	12.7318	12.7318	...
3	3	10.7579	10.822	10.7579	12.7318	12.7318	...
4	4	10.7579	10.822	10.7579	12.7318	12.7318	...
5	5	10.7579	10.822	10.7579	12.7318	12.7318	...
6	6	10.7579	10.822	10.7579	12.7318	12.7318	...
7	7	10.7579	10.822	10.7579	12.7318	12.7318	...
8	8	21.5158	21.6441	21.5158	25.4636	25.4636	...
9	9	10.7579	10.822	10.7579	12.7318	12.7318	...
10	10	10.7579	10.822	10.7579	12.7318	12.7318	...

1.2.8 Atualização dos salários dos trabalhadores

A função `Bit.update_workers_wages!(model)` transfere o salário definido pelas empresas para os trabalhadores empregados.

Cada trabalhador possui uma variável de ocupação, O_h , que identifica a empresa onde ele trabalha:

$$O_h = \begin{cases} 0, & \text{se o trabalhador está desempregado,} \\ i, & \text{se o trabalhador está empregado na empresa } i. \end{cases}$$

Para cada trabalhador empregado, o salário individual passa a ser igual ao salário oferecido por sua empresa:

$$w_h^{\text{novo}} = w_{O_h}, \quad O_h \neq 0$$

Para os trabalhadores desempregados, a função não modifica o salário armazenado:

$$w_h^{\text{novo}} = w_h^{\text{anterior}}, \quad O_h = 0$$

Definição das variáveis

Símbolo	Código	Significado
h	índice do trabalhador	Trabalhador analisado
i	índice da empresa	Empresa analisada
O_h	<code>O_h[h]</code>	Empresa que emprega o trabalhador; zero indica desemprego
w_i	<code>model.firms.w_i[i]</code>	Salário oferecido pela empresa i

Símbolo	Código	Significado
w_h	<code>model.w_act.w_h[h]</code>	Salário individual armazenado para o trabalhador h
w_{O_h}	<code>model.firms.w_i[0_h[h]]</code>	Salário oferecido pela empresa onde o trabalhador está empregado

Todos os trabalhadores de uma mesma empresa recebem o mesmo salário, pois o modelo não considera diferenças individuais de qualificação, experiência ou produtividade.

A função atualiza `model.w_act.w_h`. Ela não altera o salário definido pelas empresas, `model.firms.w_i`, nem a ocupação dos trabalhadores, `model.w_act.0_h`.

```
previous_wages = copy(model.w_act.w_h)
employers = copy(model.w_act.0_h)

Bit.update_workers_wages!(model)

employed = employers .!= 0
unemployed = .!employed

@assert all(
    model.w_act.w_h[employed] .==
    model.firms.w_i[employers[employed]]
)

@assert all(
    model.w_act.w_h[unemployed] .==
    previous_wages[unemployed]
)
```

1.2.9 Atualização dos benefícios sociais

A função `Bit.set_gov_social_benefits!(model)` atualiza os benefícios sociais pagos pelo governo de acordo com a taxa esperada de crescimento do PIB.

O benefício social geral é atualizado por:

$$sb_t^{\text{other}} = sb_{t-1}^{\text{other}} (1 + \gamma_t^e)$$

O benefício adicional destinado às pessoas inativas é atualizado por:

$$sb_t^{\text{inact}} = sb_{t-1}^{\text{inact}} (1 + \gamma_t^e)$$

Definição das variáveis

Símbolo	Código	Significado
t	<code>model.agg.t</code>	Período atual da simulação
γ_t^e	<code>model.agg.gamma_e</code>	Taxa esperada de crescimento do PIB
sb_t^{other}	<code>model.gov.sb_other</code>	Benefício social geral atualizado
sb_t^{inact}	<code>model.gov.sb_inact</code>	Benefício adicional destinado às pessoas inativas
sb_{t-1}^{other}	<code>other_benefit_before</code>	Benefício social geral antes da atualização
sb_{t-1}^{inact}	<code>inactive_benefit_before</code>	Benefício das pessoas inativas antes da atualização

Quando $\gamma_t^e > 0$, os benefícios aumentam. Quando $\gamma_t^e < 0$, eles diminuem.

As pessoas inativas recebem posteriormente tanto o benefício geral quanto o benefício específico para inativos. Essa função apenas atualiza os valores armazenados no governo; os pagamentos serão contabilizados nas etapas seguintes.

```
government = model.gov
expected_growth = model.agg.gamma_e

other_benefit_before = government.sb_other
inactive_benefit_before = government.sb_inact

expected_other_benefit =
    other_benefit_before * (1 + expected_growth)

expected_inactive_benefit =
    inactive_benefit_before * (1 + expected_growth)

Bit.set_gov_social_benefits!(model)

@assert isapprox(
    government.sb_other,
    expected_other_benefit,
)

@assert isapprox(
    government.sb_inact,
    expected_inactive_benefit,
)

DataFrame(
    benefit_type = [
        "General social benefit",
        "Inactive-person benefit",
    ],
)
```

```

],
previous_value = [
    other_benefit_before,
    inactive_benefit_before,
],
updated_value = [
    government.sb_other,
    government.sb_inact,
],
)

```

	benefit_type	previous_value	updated_value
	String	Float64	Float64
1	General social benefit	0.590286	0.586788
2	Inactive-person benefit	2.23847	2.2252

1.2.10 Lucro esperado do banco

A função `Bit.set_bank_expected_profits!(model)` calcula o lucro que o banco espera obter no período atual.

O cálculo parte do lucro realizado no período anterior e aplica dois fatores: a inflação esperada e o crescimento econômico esperado.

$$\Pi_k^e = \Pi_k (1 + \pi^e) (1 + \gamma^e)$$

Definição das variáveis

Símbolo	Código	Significado
Π_k^e	<code>model.bank.Pi_e_k</code>	Lucro esperado do banco
Π_k	<code>model.bank.Pi_k</code>	Lucro realizado pelo banco no período anterior
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada
γ^e	<code>model.agg.gamma_e</code>	Taxa esperada de crescimento do PIB
$1 + \pi^e$	<code>1 + expected_inflation</code>	Fator de atualização dos preços
$1 + \gamma^e$	<code>1 + expected_growth</code>	Fator de atualização da atividade econômica

O termo combinado:

$$(1 + \pi^e) (1 + \gamma^e)$$

representa o crescimento nominal esperado da economia. Assim, o modelo pressupõe que o lucro do banco acompanha proporcionalmente a variação esperada dos preços e da atividade econômica.

A função atualiza `model.bank.Pi_e_k`. Esse valor será utilizado posteriormente no cálculo da renda e do orçamento do proprietário do banco.

```
bank = model.bank

previous_profit = bank.Pi_k
expected_inflation = model.agg.pi_e
expected_growth = model.agg.gamma_e

nominal_adjustment =
    (1 + expected_inflation) * (1 + expected_growth)

expected_bank_profit =
    previous_profit * nominal_adjustment

Bit.set_bank_expected_profits!(model)

@assert isapprox(
    bank.Pi_e_k,
    expected_bank_profit,
)

DataFrame(
    previous_profit = [previous_profit],
    expected_inflation = [expected_inflation],
    expected_growth = [expected_growth],
    nominal_adjustment = [nominal_adjustment],
    expected_profit = [bank.Pi_e_k],
)
```

	previous_profit	expected_inflation	expected_growth	nominal_adjustment	expected_profit
	Float64	Float64	Float64	Float64	Float64
1	6476.29	0.00223145	-0.00592608	0.996292	6452.28

1.2.11 Orçamento dos trabalhadores ativos

A função `Bit.set_households_budget_act!(model)` calcula os orçamentos desejados de consumo e investimento habitacional dos trabalhadores ativos.

O grupo de trabalhadores ativos inclui tanto os empregados quanto os desempregados.

Renda esperada dos trabalhadores empregados Para um trabalhador empregado, $O_h \neq 0$, a renda esperada é:

$$Y_h^e = [w_h (1 - \tau_{SIW} - \tau_{INC} (1 - \tau_{SIW})) + sb^{\text{other}}] \bar{P}_{HH} (1 + \pi^e)$$

Renda esperada dos trabalhadores desempregados Para um trabalhador desempregado, $O_h = 0$, a renda esperada é:

$$Y_h^e = (\theta_{\text{UB}} w_h + sb^{\text{other}}) \bar{P}_{HH} (1 + \pi^e)$$

Desconto das apostas Se o trabalhador participa do mercado de apostas, uma parcela de sua renda esperada é reservada como aposta:

$$B_h = g \max(0, Y_h^e)$$

Caso não participe:

$$B_h = 0$$

A renda disponível para a definição dos orçamentos é:

$$\tilde{Y}_h^e = Y_h^e - B_h$$

Orçamentos desejados O orçamento desejado de consumo é:

$$C_h^d = \frac{\psi \tilde{Y}_h^e}{1 + \tau_{\text{VAT}}}$$

O orçamento desejado de investimento habitacional é:

$$I_h^d = \frac{\psi_H \tilde{Y}_h^e}{1 + \tau_{\text{CF}}}$$

Definição das variáveis

Símbolo	Código	Significado
h	índice do trabalhador	Trabalhador analisado
O_h	0_h[h]	Empresa do trabalhador; zero indica desemprego
Y_h^e	expected_income[h]	Renda esperada do trabalhador
$\tilde{Y}_h^e$	budget_income[h]	Renda esperada depois da aposta
w_h	model.w_act.w_h[h]	Salário do trabalhador
τ_{SIW}	model.prop.tau_SIW	Alíquota de contribuição social do trabalhador
τ_{INC}	model.prop.tau_INC	Alíquota do imposto de renda
θ_{UB}	model.prop.theta_UB	Taxa de reposição do seguro-desemprego
sb^{other}	model.gov.sb_other	Benefício social geral
$\bar{P}_{HH}$	model.agg.P_bar_HH	Índice de preços das famílias

Símbolo	Código	Significado
π^e	<code>model.agg.pi_e</code>	Inflação esperada
g	<code>model.prop.gambling_income_share</code>	Parcela da renda apostada pelos participantes
B_h	<code>gambling_stakes[h]</code>	Valor reservado para apostas
ψ	<code>model.prop.psi</code>	Parcela da renda destinada ao consumo
ψ_H	<code>model.prop.psi_H</code>	Parcela da renda destinada ao investimento habitacional
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre o consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota do imposto sobre formação de capital
C_h^d	<code>model.w_act.C_d_h[h]</code>	Orçamento desejado de consumo
I_h^d	<code>model.w_act.I_d_h[h]</code>	Orçamento desejado de investimento habitacional

A função atualiza `model.w_act.C_d_h` e `model.w_act.I_d_h`. Ela apenas define os orçamentos desejados; as compras efetivas serão determinadas posteriormente pelo mercado de bens.

```

active_workers = model.w_act

expected_income =
    Bit.households_income_act(model; expected = true)

gambling_stakes, _, _ =
    Bit.gambling_transfers(model; income_act = expected_income)

budget_income = expected_income - gambling_stakes

expected_consumption_budget =
    model.prop.psi .* budget_income ./ (1 + model.prop.tau_VAT)

expected_investment_budget =
    model.prop.psi_H .* budget_income ./ (1 + model.prop.tau_CF)

Bit.set_households_budget_act!(model)

@assert all(isapprox.(
    active_workers.C_d_h,
    expected_consumption_budget,
))

@assert all(isapprox.(
    active_workers.I_d_h,

```

```

        expected_investment_budget,
    ))

active_workers_budget = DataFrame(
    worker_id = active_workers.ID,
    employer_id = active_workers.O_h,
    expected_income = expected_income,
    gambling_stake = gambling_stakes,
    budget_income = budget_income,
    consumption_budget = active_workers.C_d_h,
    investment_budget = active_workers.I_d_h,
)

first(active_workers_budget, 10)

```

	worker_id	employer_id	expected_income	gambling_stake	budget_income	consumption_budget
	Int64	Int64	Float64	Float64	Float64	Float64
1	1	1	0.763288	0.0	0.763288	0.60227
2	2	1	0.763288	0.0	0.763288	0.60227
3	3	1	0.763288	0.0	0.763288	0.60227
4	4	2	0.763288	0.0	0.763288	0.60227
5	5	3	0.763288	0.0	0.763288	0.60227
6	6	4	0.763288	0.0	0.763288	0.60227
7	7	5	0.763288	0.0	0.763288	0.60227
8	8	6	0.763288	0.0	0.763288	0.60227
9	9	7	0.763288	0.0	0.763288	0.60227
10	10	8	0.763288	0.0	0.763288	0.60227

1.2.12 Orçamento das pessoas inativas

A função `Bit.set_households_budget_inact!(model)` calcula os orçamentos desejados de consumo e investimento habitacional das pessoas economicamente inativas.

A renda esperada de cada pessoa inativa é formada pelo benefício específico para inativos e pelo benefício social geral:

$$Y_h^e = (sb^{\text{inact}} + sb^{\text{other}}) \bar{P}_{HH} (1 + \pi^e)$$

Desconto das apostas Se a pessoa inativa participa do mercado de apostas, uma parcela de sua renda esperada é reservada como aposta:

$$B_h = g \max(0, Y_h^e)$$

Caso não participe:

$$B_h = 0$$

A renda disponível para a definição dos orçamentos é:

$$\tilde{Y}_h^e = Y_h^e - B_h$$

Orçamentos desejados O orçamento desejado de consumo é:

$$C_h^d = \frac{\psi \tilde{Y}_h^e}{1 + \tau_{\text{VAT}}}$$

O orçamento desejado de investimento habitacional é:

$$I_h^d = \frac{\psi_H \tilde{Y}_h^e}{1 + \tau_{\text{CF}}}$$

Definição das variáveis

Símbolo	Código	Significado
h	índice da pessoa inativa	Pessoa analisada
Y_h^e	<code>expected_income[h]</code>	Renda esperada
$\tilde{Y}_h^e$	<code>budget_income[h]</code>	Renda esperada depois da aposta
sb^{inact}	<code>model.gov.sb_inact</code>	Benefício específico para pessoas inativas
sb^{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços das famílias
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada
g	<code>model.prop.gambling_income_share</code>	Parcela da renda apostada pelos participantes
B_h	<code>gambling_stakes[h]</code>	Valor reservado para apostas
ψ	<code>model.prop.psi</code>	Parcela da renda destinada ao consumo
ψ_H	<code>model.prop.psi_H</code>	Parcela da renda destinada ao investimento habitacional
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota do imposto sobre formação de capital
C_h^d	<code>model.w_inact.C_d_h[h]</code>	Orçamento desejado de consumo
I_h^d	<code>model.w_inact.I_d_h[h]</code>	Orçamento desejado de investimento habitacional

Antes das apostas, todas as pessoas inativas recebem a mesma renda esperada no modelo. Diferenças no orçamento podem surgir quando apenas algumas delas participam do mercado de apostas.

A função atualiza `model.w_inact.C_d_h` e `model.w_inact.I_d_h`. As compras efetivas serão determinadas posteriormente pelo mercado de bens.

```
inactive_workers = model.w_inact

expected_income =
    Bit.households_income_inact(model; expected = true)

_, gambling_stakes, _ =
    Bit.gambling_transfers(model; income_inact = expected_income)

budget_income = expected_income - gambling_stakes

expected_consumption_budget =
    model.prop.psi .* budget_income ./ (1 + model.prop.tau_VAT)

expected_investment_budget =
    model.prop.psi_H .* budget_income ./ (1 + model.prop.tau_CF)

Bit.set_households_budget_inact!(model)

@assert all(isapprox.(
    inactive_workers.C_d_h,
    expected_consumption_budget,
))

@assert all(isapprox.(
    inactive_workers.I_d_h,
    expected_investment_budget,
))

inactive_workers_budget = DataFrame(
    worker_id = inactive_workers.ID,
    expected_income = expected_income,
    gambling_stake = gambling_stakes,
    budget_income = budget_income,
    consumption_budget = inactive_workers.C_d_h,
    investment_budget = inactive_workers.I_d_h,
)

first(inactive_workers_budget, 10)
```

	worker_id	expected_income	gambling_stake	budget_income	consumption_budget	investment_b
	Int64	Float64	Float64	Float64	Float64	Float64
1	1	2.81827	0.0	2.81827	2.22375	0.184628
2	2	2.81827	0.0	2.81827	2.22375	0.184628
3	3	2.81827	0.0	2.81827	2.22375	0.184628
4	4	2.81827	0.0	2.81827	2.22375	0.184628
5	5	2.81827	0.0	2.81827	2.22375	0.184628
6	6	2.81827	0.0	2.81827	2.22375	0.184628
7	7	2.81827	0.0	2.81827	2.22375	0.184628
8	8	2.81827	0.0	2.81827	2.22375	0.184628
9	9	2.81827	0.0	2.81827	2.22375	0.184628
10	10	2.81827	0.0	2.81827	2.22375	0.184628

1.2.13 Orçamento dos proprietários das empresas

A função `Bit.set_households_budget_firms!(model)` calcula os orçamentos desejados de consumo e investimento habitacional dos proprietários das empresas.

Cada empresa representa também uma família proprietária. Sua renda esperada é composta pelos dividendos esperados e pelo benefício social geral:

$$Y_i^e = \theta_{\text{DIV}} (1 - \tau_{\text{INC}}) (1 - \tau_{\text{FIRM}}) \max(0, \Pi_i^e) + sb^{\text{other}} \bar{P}_{HH} (1 + \pi^e)$$

O operador $\max(0, \Pi_i^e)$ impede a distribuição de dividendos quando a empresa espera prejuízo.

Receitas provenientes das apostas As apostas realizadas pelos trabalhadores ativos e pelas pessoas inativas são somadas:

$$B = \sum_h B_h^{\text{act}} + \sum_h B_h^{\text{inact}}$$

Quando a empresa pertence ao conjunto de operadoras do mercado de apostas, ela recebe uma parcela igual desse total:

$$R_i = \begin{cases} \frac{B}{N_G}, & \text{se a empresa } i \text{ é uma operadora,} \\ 0, & \text{caso contrário.} \end{cases}$$

A renda disponível para definir os orçamentos é:

$$\tilde{Y}_i^e = Y_i^e + R_i$$

Orçamentos desejados O orçamento desejado de consumo é:

$$C_i^d = \frac{\psi \tilde{Y}_i^e}{1 + \tau_{\text{VAT}}}$$

O orçamento desejado de investimento habitacional é:

$$I_i^d = \frac{\psi_H \widetilde{Y}_i^e}{1 + \tau_{CF}}$$

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa e família proprietária analisada
Y_i^e	<code>expected_income[i]</code>	Renda esperada da família proprietária
$\widetilde{Y}_i^e$	<code>budget_income[i]</code>	Renda esperada incluindo receitas das apostas
Π_i^e	<code>model.firms.Pi_e_i[i]</code>	Lucro esperado da empresa
θ_{DIV}	<code>model.prop.theta_DIV</code>	Parcela do lucro distribuída como dividendos
τ_{FIRM}	<code>model.prop.tau_FIRM</code>	Alíquota do imposto sobre o lucro da empresa
τ_{INC}	<code>model.prop.tau_INC</code>	Alíquota do imposto de renda
sb^{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços das famílias
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada
B	<code>sum(gambling_receipts)</code>	Volume total esperado de apostas
N_G	quantidade de operadoras	Número de empresas que recebem as apostas
R_i	<code>gambling_receipts[i]</code>	Receita da empresa proveniente das apostas
ψ	<code>model.prop.psi</code>	Parcela da renda destinada ao consumo
ψ_H	<code>model.prop.psi_H</code>	Parcela da renda destinada ao investimento habitacional
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota do imposto sobre formação de capital
C_i^d	<code>model.firms.C_d_h[i]</code>	Orçamento desejado de consumo do proprietário
I_i^d	<code>model.firms.I_d_h[i]</code>	Orçamento desejado de investimento habitacional

A função atualiza `model.firms.C_d_h` e `model.firms.I_d_h`. Ela não altera ainda a renda efetiva dos proprietários nem realiza as transferências das apostas.

```

firms = model.firms

active_income =
    Bit.households_income_act(model; expected = true)

inactive_income =
    Bit.households_income_inact(model; expected = true)

_, _, gambling_receipts = Bit.gambling_transfers(
    model;
    income_act = active_income,
    income_inact = inactive_income,
)

expected_income =
    Bit.households_income_firms(model; expected = true)

budget_income = expected_income + gambling_receipts

expected_consumption_budget =
    model.prop.psi .* budget_income ./ (1 + model.prop.tau_VAT)

expected_investment_budget =
    model.prop.psi_H .* budget_income ./ (1 + model.prop.tau_CF)

Bit.set_households_budget_firms!(model)

@assert all(isapprox.(
    firms.C_d_h,
    expected_consumption_budget,
))

@assert all(isapprox.(
    firms.I_d_h,
    expected_investment_budget,
))

firm_owners_budget = DataFrame(
    firm_id = firms.ID,
    expected_profit = firms.Pi_e_i,
    expected_income = expected_income,
    gambling_receipt = gambling_receipts,
    budget_income = budget_income,
    consumption_budget = firms.C_d_h,
    investment_budget = firms.I_d_h,
)

```

```
first(firm_owners_budget, 10)
```

	firm_id	expected_profit	expected_income	gambling_receipt	budget_income	consumption_budg
	Int64	Float64	Float64	Float64	Float64	Float64
1	1	3.75058	2.72783	0.0	2.72783	2.15239
2	2	1.25019	1.30134	0.0	1.30134	1.02682
3	3	1.25019	1.30134	0.0	1.30134	1.02682
4	4	1.25019	1.30134	0.0	1.30134	1.02682
5	5	1.25019	1.30134	0.0	1.30134	1.02682
6	6	1.25019	1.30134	0.0	1.30134	1.02682
7	7	1.25019	1.30134	0.0	1.30134	1.02682
8	8	2.50039	2.01459	0.0	2.01459	1.58961
9	9	1.25019	1.30134	0.0	1.30134	1.02682
10	10	1.25019	1.30134	0.0	1.30134	1.02682

1.2.14 Orçamento do proprietário do banco

A função `Bit.set_households_budget_bank!(model)` calcula os orçamentos desejados de consumo e investimento habitacional do proprietário do banco.

Sua renda esperada é formada pelos dividendos esperados do banco e pelo benefício social geral:

$$Y_k^e = \theta_{\text{DIV}} (1 - \tau_{\text{INC}}) (1 - \tau_{\text{FIRM}}) \max(0, \Pi_k^e) + sb^{\text{other}} \bar{P}_{HH} (1 + \pi^e)$$

O operador:

$$\max(0, \Pi_k^e)$$

impede a distribuição de dividendos quando o banco espera prejuízo.

Orçamentos desejados O orçamento desejado de consumo é:

$$C_k^d = \frac{\psi Y_k^e}{1 + \tau_{\text{VAT}}}$$

O orçamento desejado de investimento habitacional é:

$$I_k^d = \frac{\psi_H Y_k^e}{1 + \tau_{\text{CF}}}$$

Definição das variáveis

Símbolo	Código	Significado
Y_k^e	<code>expected_income</code>	Renda esperada do proprietário do banco
Π_k^e	<code>model.bank.Pi_e_k</code>	Lucro esperado do banco

Símbolo	Código	Significado
θ_{DIV}	<code>model.prop.theta_DIV</code>	Parcela do lucro distribuída como dividendos
τ_{FIRM}	<code>model.prop.tau_FIRM</code>	Alíquota do imposto sobre o lucro
τ_{INC}	<code>model.prop.tau_INC</code>	Alíquota do imposto de renda
sb^{other}	<code>model.gov.sb_other</code>	Benefício social geral
$\bar{P}_{HH}$	<code>model.agg.P_bar_HH</code>	Índice de preços das famílias
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada
ψ	<code>model.prop.psi</code>	Parcela da renda destinada ao consumo
ψ_H	<code>model.prop.psi_H</code>	Parcela da renda destinada ao investimento habitacional
τ_{VAT}	<code>model.prop.tau_VAT</code>	Alíquota do imposto sobre consumo
τ_{CF}	<code>model.prop.tau_CF</code>	Alíquota do imposto sobre formação de capital
C_k^d	<code>model.bank.C_d_h</code>	Orçamento desejado de consumo
I_k^d	<code>model.bank.I_d_h</code>	Orçamento desejado de investimento habitacional

Diferentemente dos proprietários das empresas, o proprietário do banco não recebe as receitas do mercado de apostas.

A função atualiza `model.bank.C_d_h` e `model.bank.I_d_h`. As compras efetivas serão determinadas posteriormente pelo mercado de bens.

```

bank = model.bank

expected_income =
    Bit.households_income_bank(model; expected = true)

expected_consumption_budget =
    model.prop.psi * expected_income / (1 + model.prop.tau_VAT)

expected_investment_budget =
    model.prop.psi_H * expected_income / (1 + model.prop.tau_CF)

Bit.set_households_budget_bank!(model)

@assert isapprox(
    bank.C_d_h,
    expected_consumption_budget,
)

@assert isapprox(

```

```

    bank.I_d_h,
    expected_investment_budget,
)

DataFrame(
    expected_bank_profit = [bank.Pi_e_k],
    expected_owner_income = [expected_income],
    consumption_budget = [bank.C_d_h],
    investment_budget = [bank.I_d_h],
)

```

	expected_bank_profit	expected_owner_income	consumption_budget	investment_budget
	Float64	Float64	Float64	Float64
1	6452.28	3681.67	2905.01	241.191

1.2.15 Orçamento de consumo do governo

A função `Bit.set_gov_expenditure!(model)` calcula o consumo público agregado e distribui seu orçamento igualmente entre os governos locais.

Consumo público agregado O consumo do governo segue um processo autorregressivo em logaritmos:

$$\log C_{G,t} = \alpha_G \log C_{G,t-1} + \beta_G + \varepsilon_{G,t}$$

Consequentemente:

$$C_{G,t} = \exp(\alpha_G \log C_{G,t-1} + \beta_G + \varepsilon_{G,t})$$

O choque do governo é sorteado de uma distribuição normal:

$$\varepsilon_{G,t} \sim \mathcal{N}(0, \sigma_G^2)$$

Distribuição entre os governos locais O orçamento de cada governo local é:

$$C_j^d = \frac{C_{G,t}}{J} \left(\sum_g c_{G,g} \bar{P}_g \right) (1 + \pi^e)$$

Como o orçamento é dividido igualmente, todos os governos locais recebem o mesmo valor.

Definição das variáveis

Símbolo	Código	Significado
$C_{G,t}$	<code>model.gov.C_G</code>	Consumo agregado do governo no período atual

Símbolo	Código	Significado
$C_{G,t-1}$	<code>government_consumption_before</code>	Consumo agregado do governo no período anterior
α_G	<code>model.gov.alpha_G</code>	Coefficiente autorregressivo do consumo público
β_G	<code>model.gov.beta_G</code>	Constante do processo de consumo público
$\varepsilon_{G,t}$	<code>implied_government_shock</code>	Choque aleatório do consumo público
σ_G	<code>model.gov.sigma_G</code>	Escala do choque aleatório
J	<code>local_government_count</code>	Número de governos locais
C_j^d	<code>model.gov.C_d_j[j]</code>	Orçamento de consumo do governo local j
g	índice do produto	Produto analisado
$c_{G,g}$	<code>model.prop.c_G_g[g]</code>	Coefficiente de consumo público do produto g
$\bar{P}_g$	<code>model.agg.P_bar_g[g]</code>	Índice de preços do produto g
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada

A função atualiza `model.gov.C_G` e `model.gov.C_d_j`. O consumo efetivamente realizado será determinado posteriormente pelo mercado de bens.

Como a função sorteia um novo choque $\varepsilon_{G,t}$, executá-la novamente produz um novo orçamento público.

```

government = model.gov

government_consumption_before = government.C_G
local_government_count = length(government.C_d_j)

Bit.set_gov_expenditure!(model)

government_consumption_after = government.C_G

implied_government_shock =
    log(government_consumption_after) -
    government.alpha_G * log(government_consumption_before) -
    government.beta_G

government_price_factor =
    sum(model.prop.c_G_g .* model.agg.P_bar_g)

expected_local_budget =
    government_consumption_after /
    local_government_count *
    government_price_factor *

```

```

        (1 + model.agg.pi_e)

    @assert all(isapprox.(
        government.C_d_j,
        expected_local_budget,
    ))

    local_government_budget = DataFrame(
        local_government_id = 1:local_government_count,
        aggregate_consumption = fill(
            government_consumption_after,
            local_government_count,
        ),
        implied_shock = fill(
            implied_government_shock,
            local_government_count,
        ),
        consumption_budget = government.C_d_j,
    )

    local_government_budget

```

	local_government_id	aggregate_consumption	implied_shock	consumption_budget
	Int64	Float64	Float64	Float64
1	1	14928.2	0.00975426	95.9068
2	2	14928.2	0.00975426	95.9068
3	3	14928.2	0.00975426	95.9068
4	4	14928.2	0.00975426	95.9068
5	5	14928.2	0.00975426	95.9068
6	6	14928.2	0.00975426	95.9068
7	7	14928.2	0.00975426	95.9068
8	8	14928.2	0.00975426	95.9068
9	9	14928.2	0.00975426	95.9068
10	10	14928.2	0.00975426	95.9068
11	11	14928.2	0.00975426	95.9068
12	12	14928.2	0.00975426	95.9068
13	13	14928.2	0.00975426	95.9068
14	14	14928.2	0.00975426	95.9068
15	15	14928.2	0.00975426	95.9068
16	16	14928.2	0.00975426	95.9068
17	17	14928.2	0.00975426	95.9068
18	18	14928.2	0.00975426	95.9068
19	19	14928.2	0.00975426	95.9068
20	20	14928.2	0.00975426	95.9068
21	21	14928.2	0.00975426	95.9068
22	22	14928.2	0.00975426	95.9068
23	23	14928.2	0.00975426	95.9068
24	24	14928.2	0.00975426	95.9068
25	25	14928.2	0.00975426	95.9068
26	26	14928.2	0.00975426	95.9068
27	27	14928.2	0.00975426	95.9068
28	28	14928.2	0.00975426	95.9068
29	29	14928.2	0.00975426	95.9068
30	30	14928.2	0.00975426	95.9068
...	...	...	...	...

1.2.16 Exportações e importações

A função `Bit.set_rotw_import_export!(model)` calcula a demanda externa pelas exportações nacionais e a oferta de produtos importados.

Demanda agregada por exportações A demanda externa segue um processo autorregressivo em logaritmos:

$$C_{E,t} = \exp(\alpha_E \log C_{E,t-1} + \beta_E + \varepsilon_{E,t})$$

O orçamento é dividido igualmente entre os L parceiros externos:

$$C_l^d = \frac{C_{E,t}}{L} \left(\sum_g c_{E,g} \bar{P}_g \right) (1 + \pi^e)$$

Oferta agregada de importações A oferta de importações também segue um processo autor-regressivo:

$$Y_{I,t} = \exp(\alpha_I \log Y_{I,t-1} + \beta_I + \varepsilon_{I,t})$$

A oferta importada de cada produto é:

$$Y_{m,g} = c_{I,g} Y_{I,t}$$

O preço dos produtos importados acompanha o índice de preços doméstico e a inflação esperada:

$$P_{m,g} = \bar{P}_g (1 + \pi^e)$$

Definição das variáveis

Símbolo	Código	Significado
$C_{E,t}$	<code>model.rotw.C_E</code>	Demanda externa agregada por exportações
$C_{E,t-1}$	<code>exports_before</code>	Demanda externa do período anterior
α_E	<code>model.rotw.alpha_E</code>	Coefficiente autorregressivo das exportações
β_E	<code>model.rotw.beta_E</code>	Constante do processo das exportações
$\varepsilon_{E,t}$	<code>model.agg.epsilon_E</code>	Choque sobre a demanda por exportações
L	<code>export_partner_count</code>	Número de parceiros externos
C_l^d	<code>model.rotw.C_d_l[1]</code>	Orçamento de exportação do parceiro l
$c_{E,g}$	<code>model.prop.c_E_g[g]</code>	Coefficiente de demanda externa pelo produto g
$Y_{I,t}$	<code>model.rotw.Y_I</code>	Oferta agregada de importações
$Y_{I,t-1}$	<code>imports_before</code>	Oferta agregada de importações do período anterior
α_I	<code>model.rotw.alpha_I</code>	Coefficiente autorregressivo das importações
β_I	<code>model.rotw.beta_I</code>	Constante do processo das importações

Símbolo	Código	Significado
$\varepsilon_{I,t}$	<code>model.agg.epsilon_I</code>	Choque sobre a oferta de importações
$Y_{m,g}$	<code>model.rotw.Y_m[g]</code>	Oferta importada do produto g
$c_{I,g}$	<code>model.prop.c_I_g[g]</code>	Participação do produto g nas importações
$P_{m,g}$	<code>model.rotw.P_m[g]</code>	Preço do produto importado g
$\bar{P}_g$	<code>model.agg.P_bar_g[g]</code>	Índice de preços doméstico do produto g
π^e	<code>model.agg.pi_e</code>	Taxa de inflação esperada

Os choques $\varepsilon_{E,t}$ e $\varepsilon_{I,t}$ já foram sorteados anteriormente por `Bit.set_epsilon!(model)`.

A função atualiza `model.rotw.C_E`, `model.rotw.C_d_l`, `model.rotw.Y_I`, `model.rotw.Y_m` e `model.rotw.P_m`. As exportações e importações efetivamente realizadas serão determinadas posteriormente pelo mercado de bens.

```
rest_of_world = model.rotw

exports_before = rest_of_world.C_E
imports_before = rest_of_world.Y_I
export_partner_count = length(rest_of_world.C_d_l)

expected_exports = exp(
    rest_of_world.alpha_E * log(exports_before) +
    rest_of_world.beta_E +
    model.agg.epsilon_E
)

export_price_factor =
    sum(model.prop.c_E_g .* model.agg.P_bar_g)

expected_export_budget =
    fill(
        expected_exports /
        export_partner_count *
        export_price_factor *
        (1 + model.agg.pi_e),
        export_partner_count,
    )

expected_imports = exp(
    rest_of_world.alpha_I * log(imports_before) +
    rest_of_world.beta_I +
    model.agg.epsilon_I
)
```

```

expected_import_supply =
    model.prop.c_I_g .* expected_imports

expected_import_prices =
    model.agg.P_bar_g .* (1 + model.agg.pi_e)

Bit.set_rotw_import_export!(model)

@assert isapprox(rest_of_world.C_E, expected_exports)
@assert isapprox(rest_of_world.Y_I, expected_imports)
@assert all(isapprox.(rest_of_world.C_d_l, expected_export_budget))
@assert all(isapprox.(rest_of_world.Y_m, expected_import_supply))
@assert all(isapprox.(rest_of_world.P_m, expected_import_prices))

trade_summary = DataFrame(
    metric = [
        "Aggregate export demand",
        "Aggregate import supply",
        "Export shock",
        "Import shock",
    ],
    value = [
        rest_of_world.C_E,
        rest_of_world.Y_I,
        model.agg.epsilon_E,
        model.agg.epsilon_I,
    ],
)

imports_by_product = DataFrame(
    product_id = eachindex(rest_of_world.Y_m),
    import_supply = rest_of_world.Y_m,
    import_price = rest_of_world.P_m,
)

display(trade_summary)
imports_by_product

```

	metric	value
	String	Float64
1	Aggregate export demand	34686.4
2	Aggregate import supply	33307.1
3	Export shock	0.0127531
4	Import shock	0.00277058

	product_id	import_supply	import_price
	Int64	Float64	Float64
1	1	559.916	1.00223
2	2	136.14	1.00223
3	3	12.2856	1.00223
4	4	1937.95	1.00223
5	5	1630.41	1.00223
6	6	1561.82	1.00223
7	7	336.454	1.00223
8	8	535.123	1.00223
9	9	12.4475	1.00223
10	10	1365.89	1.00223
11	11	2906.92	1.00223
12	12	1000.47	1.00223
13	13	1006.14	1.00223
14	14	435.707	1.00223
15	15	1903.97	1.00223
16	16	1092.04	1.00223
17	17	2035.91	1.00223
18	18	1332.64	1.00223
19	19	3001.95	1.00223
20	20	2815.55	1.00223
21	21	421.093	1.00223
22	22	977.131	1.00223
23	23	178.891	1.00223
24	24	217.497	1.00223
25	25	0.0672703	1.00223
26	26	388.693	1.00223
27	27	304.888	1.00223
28	28	13.6626	1.00223
29	29	81.4499	1.00223
30	30	0.0	1.00223
...	...	...	...

1.2.17 Pareamento nos mercados de bens

A função `Bit.search_and_matching!(model; parallel = false)` realiza as compras e vendas de todos os produtos da economia.

Ela reúne dois mercados:

1. o mercado entre empresas, no qual são comprados insumos intermediários e bens de capital;
2. o mercado varejista, no qual compram as famílias, o governo e os parceiros externos.

O algoritmo é executado separadamente para cada produto g .

Oferta disponível A oferta de uma empresa doméstica é formada pela produção atual e pelo estoque acumulado:

$$S_i^f = Y_i + S_i$$

A oferta de um produto importado é:

$$S_{m,g}^f = Y_{m,g}$$

Demanda das empresas A demanda da empresa i pelo produto g combina insumos intermediários e investimento produtivo:

$$D_{i,g}^F = a_{g,G_i} DM_i^d + b_g^{CF} I_i^d$$

A primeira parcela representa os insumos necessários à produção. A segunda representa os bens de capital desejados pela empresa.

Demanda do mercado varejista A demanda da família h pelo produto g é:

$$D_{h,g}^R = b_g^{HH} C_h^d + b_g^{CFH} I_h^d$$

A demanda dos parceiros externos é:

$$D_{l,g}^R = c_g^E C_l^d$$

A demanda dos governos locais é:

$$D_{j,g}^R = c_g^G C_j^d$$

Os coeficientes de consumo são normalizados pelo índice de preços de cada produto antes do pareamento.

Escolha dos fornecedores Os compradores são visitados em ordem aleatória. Para cada compra, um fornecedor é sorteado utilizando um peso que combina preço e tamanho da oferta:

$$\omega_f = \frac{\exp(-2P_f)}{\sum_r \exp(-2P_r)} + \frac{S_f}{\sum_r S_r}$$

Assim, fornecedores maiores e com preços menores possuem maior probabilidade de serem escolhidos.

A compra continua até que a demanda do comprador seja atendida ou a oferta disponível termine. Quando necessário, uma segunda rodada considera a capacidade produtiva adicional das empresas.

Vendas realizadas As vendas de cada empresa doméstica são limitadas por sua oferta e pela demanda recebida:

$$Q_i = \min(Y_i + S_i, Q_i^d)$$

As vendas dos produtos importados são:

$$Q_{m,g} = \min(Y_{m,g}, Q_{m,g}^d)$$

Definição das variáveis

Símbolo	Código	Significado
g	índice do produto	Produto negociado
i	índice da empresa	Empresa doméstica
f	índice do fornecedor	Empresa doméstica ou fornecedor estrangeiro
h	índice da família	Família compradora
j	índice do governo local	Governo comprador
l	índice do parceiro externo	Comprador estrangeiro
Y_i	<code>model.firms.Y_i[i]</code>	Produção atual da empresa
S_i	<code>model.firms.S_i[i]</code>	Estoque de produtos da empresa
P_f	preços domésticos e importados	Preço do fornecedor
DM_i^d	<code>model.firms.DM_d_i[i]</code>	Demanda desejada por insumos
I_i^d	<code>model.firms.I_d_i[i]</code>	Investimento produtivo desejado
a_{g,G_i}	<code>model.prop.a_sg[g, G_i]</code>	Necessidade do produto g como insumo
b_g^{CF}	<code>model.prop.b_CF_g[g]</code>	Participação do produto g no investimento empresarial
C_h^d	orçamento de consumo	Consumo desejado da família
I_h^d	orçamento de investimento	Investimento habitacional desejado
b_g^{HH}	<code>model.prop.b_HH_g[g]</code>	Participação do produto g no consumo familiar
b_g^{CFH}	<code>model.prop.b_CFH_g[g]</code>	Participação do produto g no investimento habitacional
c_g^G	<code>model.prop.c_G_g[g]</code>	Participação do produto g no consumo público
c_g^E	<code>model.prop.c_E_g[g]</code>	Participação do produto g nas exportações
Q_i^d	<code>model.firms.Q_d_i[i]</code>	Demanda total recebida pela empresa

Símbolo	Código	Significado
Q_i	<code>model.firms.Q_i[i]</code>	Vendas realizadas pela empresa
$Y_{m,g}$	<code>model.rotw.Y_m[g]</code>	Oferta disponível de importações
$Q_{m,g}^d$	<code>model.rotw.Q_d_m[g]</code>	Demanda por importações
$Q_{m,g}$	<code>model.rotw.Q_m[g]</code>	Vendas realizadas de importações

A função também atualiza o consumo e o investimento realizados pelas famílias, as compras de insumos e capital das empresas, o consumo público, as exportações e diversos preços médios.

O investimento habitacional realizado é acrescentado ao estoque de capital das famílias:

$$K_h^{\text{nov}} = K_h^{\text{anterior}} + I_h$$

Como o pareamento contém sorteios, executar novamente essa função pode produzir fornecedores e resultados diferentes. O argumento `parallel = false` mantém o processamento sequencial dos produtos.

```
firms = model.firms
rest_of_world = model.rotw
active_workers = model.w_act
inactive_workers = model.w_inact
bank = model.bank

domestic_supply_before =
  copy(firms.Y_i .+ firms.S_i)

import_supply_before =
  copy(rest_of_world.Y_m)

active_capital_before =
  copy(active_workers.K_h)

inactive_capital_before =
  copy(inactive_workers.K_h)

firm_owner_capital_before =
  copy(firms.K_h)

bank_owner_capital_before =
  bank.K_h

Bit.search_and_matching!(model; parallel = false)
```

```

expected_domestic_sales =
    min.(domestic_supply_before, firms.Q_d_i)

expected_import_sales =
    min.(import_supply_before, rest_of_world.Q_d_m)

@assert all(isapprox.(
    firms.Q_i,
    expected_domestic_sales,
))

@assert all(isapprox.(
    rest_of_world.Q_m,
    expected_import_sales,
))

@assert all(isapprox.(
    active_workers.K_h,
    active_capital_before .+ active_workers.I_h,
))

@assert all(isapprox.(
    inactive_workers.K_h,
    inactive_capital_before .+ inactive_workers.I_h,
))

@assert all(isapprox.(
    firms.K_h,
    firm_owner_capital_before .+ firms.I_h,
))

@assert isapprox(
    bank.K_h,
    bank_owner_capital_before + bank.I_h,
)

market_summary = DataFrame(
    metric = [
        "Domestic supply",
        "Domestic firm sales",
        "Import supply",
        "Import sales",
        "Household consumption",
        "Household investment",
        "Government consumption",
        "Exports",
    ],

```

```

value = [
    sum(domestic_supply_before),
    sum(firms.Q_i),
    sum(import_supply_before),
    sum(rest_of_world.Q_m),
    sum(active_workers.C_h) +
    sum(inactive_workers.C_h) +
    sum(firms.C_h) +
    bank.C_h,
    sum(active_workers.I_h) +
    sum(inactive_workers.I_h) +
    sum(firms.I_h) +
    bank.I_h,
    model.gov.C_j,
    rest_of_world.C_l,
],
)

market_summary

```

	metric	value
	String	Float64
1	Domestic supply	1.33838e5
2	Domestic firm sales	1.33827e5
3	Import supply	33307.1
4	Import sales	33286.0
5	Household consumption	34903.0
6	Household investment	2895.83
7	Government consumption	14721.4
8	Exports	34417.7

1.2.18 Inflação e índice geral de preços

A função `Bit.set_inflation_priceindex!(model)` calcula o novo índice geral de preços e a inflação realizada no período.

Índice geral de preços O índice é uma média dos preços das empresas ponderada por suas produções:

$$\bar{P}_t = \frac{\sum_i P_i Y_i}{\sum_i Y_i}$$

Empresas com maior produção possuem maior peso no índice.

Inflação realizada A inflação é calculada como a variação logarítmica entre o índice atual e o índice anterior:

$$\pi_t = \log \left(\frac{\bar{P}_t}{\bar{P}_{t-1}} \right)$$

Definição das variáveis

Símbolo	Código	Significado
i	índice da empresa	Empresa analisada
P_i	<code>model.firms.P_i[i]</code>	Preço definido pela empresa
Y_i	<code>model.firms.Y_i[i]</code>	Produção realizada pela empresa
$P_i Y_i$	<code>price[i] * production[i]</code>	Valor nominal da produção da empresa
$\bar{P}_{t-1}$	<code>price_index_before</code>	Índice geral de preços anterior
$\bar{P}_t$	<code>model.agg.P_bar</code>	Novo índice geral de preços
π_t	<code>model.agg.pi_[inflation_position]</code>	Inflação realizada no período
π^e	<code>model.agg.pi_e</code>	Inflação esperada calculada no início do período

Quando:

$$\bar{P}_t > \bar{P}_{t-1},$$

a inflação é positiva. Quando o novo índice é menor que o anterior, a inflação é negativa.

A função acrescenta a inflação realizada ao histórico `model.agg.pi_` e atualiza `model.agg.P_bar`.

Esse índice utiliza a produção doméstica, Y_i , como ponderação. As quantidades importadas e as vendas efetivamente realizadas não entram diretamente nesse cálculo.

```

aggregate = model.agg
firms = model.firms

price_index_before = aggregate.P_bar
inflation_history_length_before = length(aggregate.pi_)

prices = copy(firms.P_i)
production = copy(firms.Y_i)

nominal_production =
    sum(prices .* production)

total_production =
    sum(production)

expected_price_index =
    nominal_production / total_production

```

```

expected_inflation =
    log(expected_price_index / price_index_before)

inflation_position =
    model.prop.T_prime + aggregate.t

Bit.set_inflation_priceindex!(model)

@assert isapprox(
    aggregate.P_bar,
    expected_price_index,
)

@assert isapprox(
    aggregate.pi_[inflation_position],
    expected_inflation,
)

@assert length(aggregate.pi_) ==
    inflation_history_length_before + 1

price_contributions = DataFrame(
    firm_id = firms.ID,
    price = prices,
    production = production,
    nominal_production = prices .* production,
    production_weight = production ./ total_production,
)

inflation_summary = DataFrame(
    metric = [
        "Previous price index",
        "Current price index",
        "Realized inflation",
        "Expected inflation",
    ],
    value = [
        price_index_before,
        aggregate.P_bar,
        aggregate.pi_[inflation_position],
        aggregate.pi_e,
    ],
)

display(inflation_summary)
first(price_contributions, 10)

```

	metric	value
	String	Float64
1	Previous price index	1.0
2	Current price index	1.00223
3	Realized inflation	0.00222896
4	Expected inflation	0.00223145

	firm_id	price	production	nominal_production	production_weight
	Int64	Float64	Float64	Float64	Float64
1	1	1.00223	32.2737	32.3457	0.00024114
2	2	1.00223	10.7579	10.7819	8.038e-5
3	3	1.00223	10.7579	10.7819	8.038e-5
4	4	1.00223	10.7579	10.7819	8.038e-5
5	5	1.00223	10.7579	10.7819	8.038e-5
6	6	1.00223	10.7579	10.7819	8.038e-5
7	7	1.00223	10.7579	10.7819	8.038e-5
8	8	1.00223	21.5158	21.5638	0.00016076
9	9	1.00223	10.7579	10.7819	8.038e-5
10	10	1.00223	10.7579	10.7819	8.038e-5

